CM3604 – Manura's Contribution Guide

Fact-checking coursework · Repository: SandaliBalasooriya/CM3604-factchecker · Prepared 9 Nov 2026 · Deadline 11 Nov 2026, 23:59

> **Goal:** Manura (Member D) adds one or two self-contained improvements on his own branch, with a few clear commits, then merges them into `main` through a pull request. Recommended: **Task 1 (parsing mode)** plus **Task 3 (Part 1 unit tests)**. Allow about 4–6 hours in total, and finish by **11 Nov, 18:00** so there's time to review.

1. Where the project stands

Item	Value
Repository	https://github.com/SandaliBalasooriya/CM3604-factchecker (private)
`main` head	`beb8c12` – "Count IR facts as NS in threshold_sweep evaluate()"
Existing branches	`main`, `member-b/setup-and-preprocessing`, `member-c/sachinthana-evaluation`
Manura's branch (to create)	`member-d/manura-<feature>`, e.g. `member-d/manura-parsing-mode`
`--mode word_overlap`	81.0% (179/221), target 75%, about 3 s
`--mode entailment`	84.6% (187/221), target 83%, about 2 min on CPU
Tests	`python -m unittest discover -s tests` → 3/3 pass
Team	A Sandali (Part 1) · B Dilshan (Part 2) · C Sachinthana (evaluation, tests) · D Manura

2. Suggested contributions

Each task is independent, so it won't clash with anyone else's work. None of them may change the two required modes' results (81.0% and 84.6%) unless the whole group agrees, because the report's numbers depend on them.

Recommended ~2–3 h **Task 1 – Implement the dependency-parsing fact-checker (`--mode parsing`)**

Why: the framework already has `DependencyRecallThresholdFactChecker` in `factcheck.py`, but its `predict()` still raises `Exception("Implement me")`, so `python factchecking_main.py --mode parsing` crashes. This is the only unfinished mode in the project, and it's a natural third method to compare with word overlap and entailment.

What to do:

1. Use the existing `get_dependencies(sent)` helper, which returns `(head, relation, child)` triples from spaCy.
2. In `predict()`: get the fact's triples. Split each passage into sentences (`nltk.sent_tokenize`) and collect their triples. Score = **recall**, meaning the fraction of the fact's triples found in the passage, max over passages. Return "S" if score ≥ threshold, otherwise "NS".
3. Lowercase the head and child words before comparing. If the fact has no triples, return "NS".
4. Add a `threshold` argument to `__init__`. Try values 0.1–0.9 with a short script (copy the style of `threshold_sweep.py`) and set the best as the default.
5. Keep spaCy fast: parse each passage once per fact, not once per triple.

Done when: `python factchecking_main.py --mode parsing` runs without errors and prints an accuracy; `word_overlap` and `entailment` still give 179/221 and 187/221; you send the group the accuracy and chosen threshold for the report.

Example commits: "Implement dependency-recall predict() for parsing mode" · "Tune parsing threshold on dev set".

Recommended ~1–2 h **Task 3 – Unit tests for Part 1 (word overlap)**

Why: the only tests so far check the evaluation labels. Part 1's preprocessing and scorers have no tests, and they're cheap to test because no model is needed.

What to do: create `tests/test_word_overlap.py` (Python `unittest`, same style as `tests/test_evaluation_labels.py`) covering:

- `preprocess()`: lowercasing; punctuation split off ("Ohio." → "ohio"); stop words removed; stemming ("inventing" → "invent"); each toggle turned off.
- `_compute_score()` for `recall`, `jaccard`, `f1` and `rouge` on small hand-made sets with known answers; an empty fact → 0.0; an unknown scorer raises `ValueError`.
- `predict()` at the threshold boundary: a score exactly 0.65 → "S", just below → "NS".

Done when: `python -m unittest discover -s tests -v` shows all old and new tests passing.

Example commits: "Add unit tests for word-overlap preprocessing" · "Add scorer and threshold tests".

Alternative ~1 h

Task 2 – Make results reproducible across Windows and Mac

Why: `evaluate_entailment.py` caches its results using SHA-256 hashes of the code and data files. Windows and Mac store line endings differently, so the hashes never match across machines. The script then re-scores everything and rewrites both files in `results/`, which shows them as modified in git even though the numbers are the same.

What to do: in `file_hash()`, hash the text with line endings normalised (`path.read_bytes().replace(b"\r\n", b"\n")`). Write the CSV with `newline=""` and `lineterminator="\n"`. Add a `.gitattributes` file containing `* text=auto eol=lf`.

Done when: running `python evaluate_entailment.py` twice on any machine reuses the cache the second time instead of re-scoring, and `git status` stays clean.

Stretch ~2 h + 5 min run

Task 4 – Sliding-window premises experiment

Why: the report's error analysis shows most false negatives happen when the evidence is split across neighbouring sentences or list items. Scoring windows of 2 sentences joined together may recover some of them.

What to do: add a script `report/window_test.py` that reuses `EntailmentFactChecker`'s logic but also scores each pair of adjacent sentences. Report accuracy and model calls for window size 1 (current) and 2. Don't change the submitted defaults in `factcheck.py`.

Done when: the script runs and prints a small comparison table the group can quote in the report's discussion.

3. One-time setup (Windows, PowerShell)

3.1 Get access

1. **Sandali**: on GitHub, open the repo → **Settings** → **Collaborators** → **Add people** → enter Manura's GitHub username → choose **Write** access.
2. **Manura**: accept the invitation from the email or from github.com/notifications. Until then, the repo shows "404 – not found" for him because it's private.

3.2 Install the tools

- **Git**: https://git-scm.com/download/win (keep the defaults; it includes Git Credential Manager for sign-in).
- **Python 3.10.11**: https://www.python.org/downloads/release/python-31011/. Tick **"Add to PATH"**. Keeping a newer Python alongside is fine, because the `py` launcher picks the version.

```
# tell git who you are (once per computer) — use the email on your GitHub account
git config --global user.name "Manura <Surname>"
git config --global user.email "manura@example.com"
```

3.3 Clone and set up the project

```
# pick a folder, e.g. Documents
cd $HOME\Documents
git clone https://github.com/SandaliBalasooriya/CM3604-factchecker.git
cd CM3604-factchecker
# a browser window opens the first time: sign in to GitHub and click Authorize

py -3.10 -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

If `venv\Scripts\activate` is blocked, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then try again. You don't need to download NLTK data by hand; `factcheck.py` fetches it on first run.

3.4 Check that everything works before changing anything

```
python factchecking_main.py --mode word_overlap      # expect Accuracy: 179/221
python factchecking_main.py --mode entailment        # expect Accuracy: 187/221 (first
run downloads the model, ~700 MB)
python -m unittest discover -s tests -v               # expect OK
```

4. Branch, commit and push

```
# 1. start from the latest main
git checkout main
git pull

# 2. create your branch (one per task)
git checkout -b member-d/manura-parsing-mode

# 3. edit code, then check what changed
git status
git diff

# 4. stage only the files you meant to change, then commit
git add factcheck.py
git commit -m "Implement dependency-recall predict() for parsing mode"

# 5. push the branch to GitHub (first time uses -u)
git push -u origin member-d/manura-parsing-mode

# later commits on the same branch: just
git add tests/test_word_overlap.py
git commit -m "Add unit tests for word-overlap preprocessing"
git push
```

Good commits: one logical change each, a short message in the imperative ("Add…", "Fix…", "Tune…"), and both modes plus the tests run before every push. Three to five commits for a task is about right.

If `main` changed while you were working:

```
git fetch origin
git merge origin/main        # fix any conflicts, then: git add <file> ; git commit
git push
```

5. Getting the work into `main`

Recommended: a pull request on GitHub

1. After pushing, open the repo on GitHub. Click the yellow **"Compare & pull request"** banner, or go to **Pull requests** → **New pull request**.
2. Set **base: main** and **compare: member-d/manura-parsing-mode**.
3. Title, e.g. "Add parsing mode (dependency recall)". In the description, list what changed, the accuracy you got, and that both required modes still give 179/221 and 187/221.
4. Under **Reviewers**, add Sandali or another member and click **Create pull request**.
5. Once a teammate has looked at it, click **Merge pull request** → **Confirm merge**, then **Delete branch**.
6. Back on your computer:

   ```
   git checkout main
   git pull
   ```

Alternative: merge from the terminal (only if the group agrees)

```
git checkout main
git pull
git merge member-d/manura-parsing-mode
python factchecking_main.py --mode word_overlap     # re-check before pushing main
git push origin main
```

6. Rules and checklist

> **Do not change:** `factchecking_main.py` (the brief requires the driver to be unmodified), `data/`, `requirements.txt`, or the default thresholds in `factcheck.py` (0.65, 0.38, 0.05) without telling the group, because the report quotes results that depend on them. **Never** use `git push --force` on `main`.

Don't commit: `venv/`, `.idea/`, `__pycache__/`, or the two `results/` files if `evaluate_entailment.py` rewrote them (undo with `git restore results/`).

Before every push:

- ☐ `python factchecking_main.py --mode word_overlap` → 179/221
- ☐ `python factchecking_main.py --mode entailment` → 187/221 (only needed if `factcheck.py` changed)
- ☐ `python -m unittest discover -s tests` → OK
- ☐ `git status` shows only the files you meant to change

Academic integrity: the brief's GenAI category is "❸ Editing", so the code and ideas should be your own. If you use any AI tool, tell the group so it goes into the report's GenAI acknowledgement.

Troubleshooting

Problem	Fix
"Repository not found" when cloning	The collaborator invite hasn't been accepted yet, or git is signed in to a different GitHub account. Accept the invite. In Windows **Credential Manager** → Windows Credentials, remove `git:https://github.com` and clone again to sign in fresh.
"Password authentication is not supported"	Don't type your password. Let the browser sign-in pop-up handle it, or use a personal access token (GitHub → Settings → Developer settings → Tokens, scope **repo**).
`py -3.10` not found	Python 3.10 isn't installed; install 3.10.11 from the link above.
push rejected ("fetch first")	Someone pushed before you: `git pull`, re-run the checks, then `git push`.
Merge conflict	Open the file, keep the right lines between `<<<<<<<` and `>>>>>>>`, delete the markers, then `git add <file>` and `git commit`. Ask the group if unsure.
Entailment is very slow	It's about 2 minutes on CPU, which is normal. The first run also downloads the model. Add `--cuda` only if PyTorch with CUDA is installed.